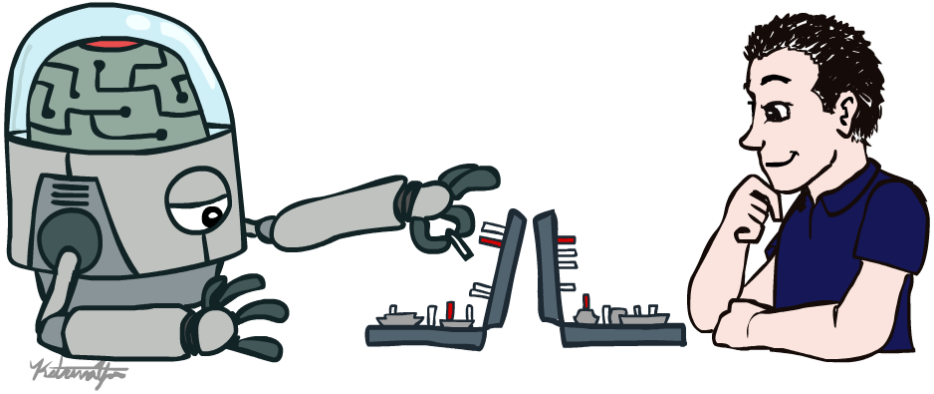
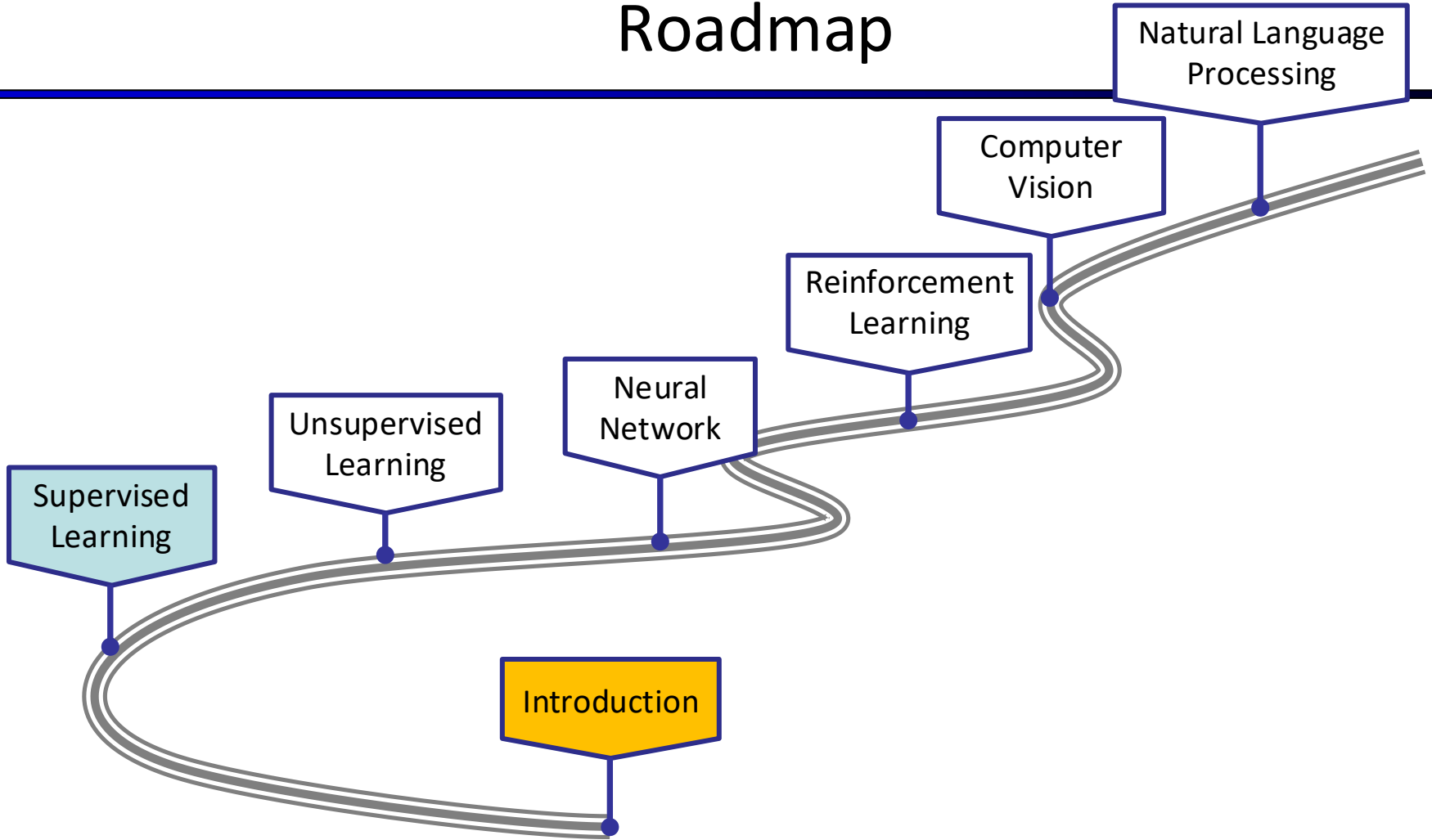
Artificial Intelligence

Supervised Learning



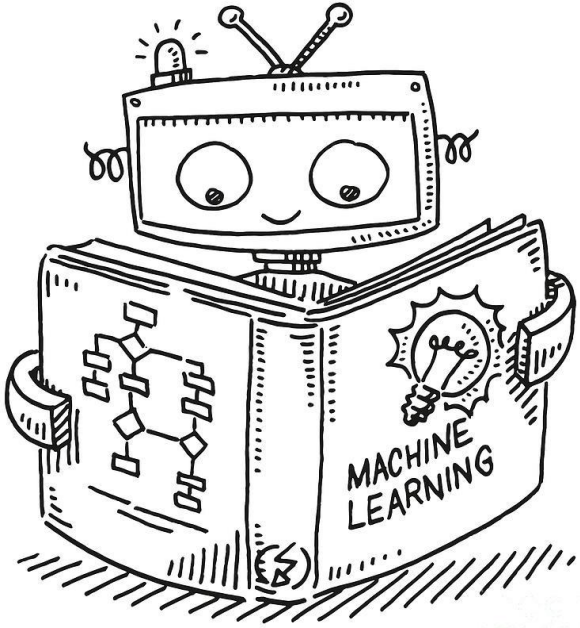
Yan Lyu

Roadmap


Natural Language Processing
Computer Vision
Reinforcement Learning
Neural Network
Unsupervised Learning
Supervised Learning
Introduction

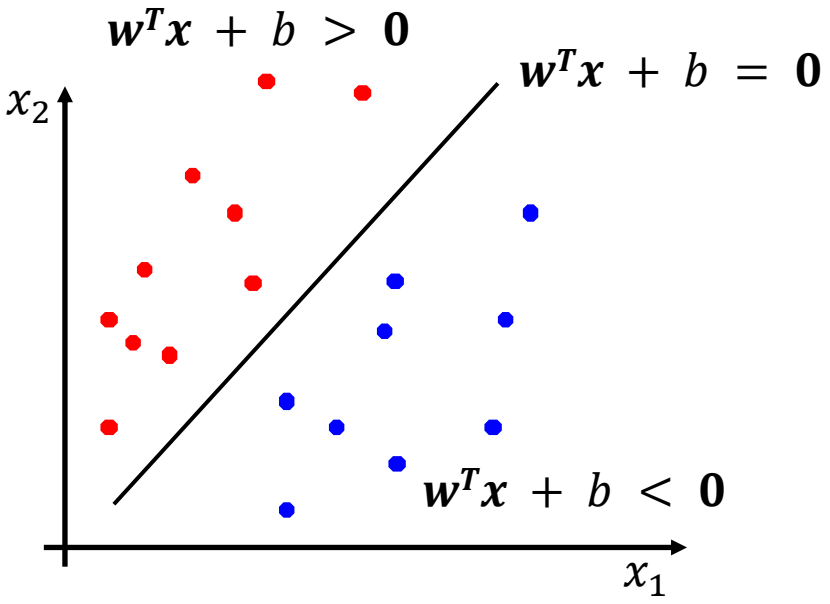
Today

- **Classification: Support Vector Machine**
- Classification: Decision Tree
- Discussion



Revisited: Linear Separators

- Binary classification can be viewed as the task of separating classes in feature space:

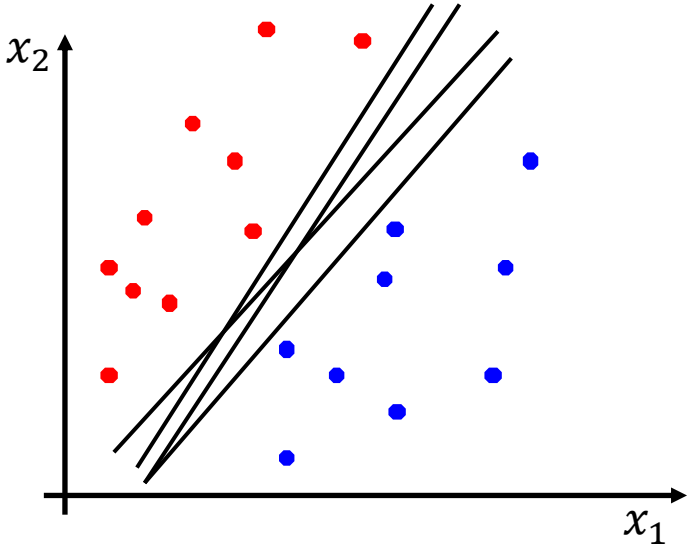


$$\boldsymbol{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} \qquad \boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

$$f(\boldsymbol{x}) = \operatorname{sign}(\boldsymbol{w}^T\boldsymbol{x} + b)$$

Revisited: Linear Separators

- Which of the linear separators is optimal?



$$\boldsymbol{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} \qquad \boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

$$f(\boldsymbol{x}) = \operatorname{sign}(\boldsymbol{w}^T\boldsymbol{x} + b)$$

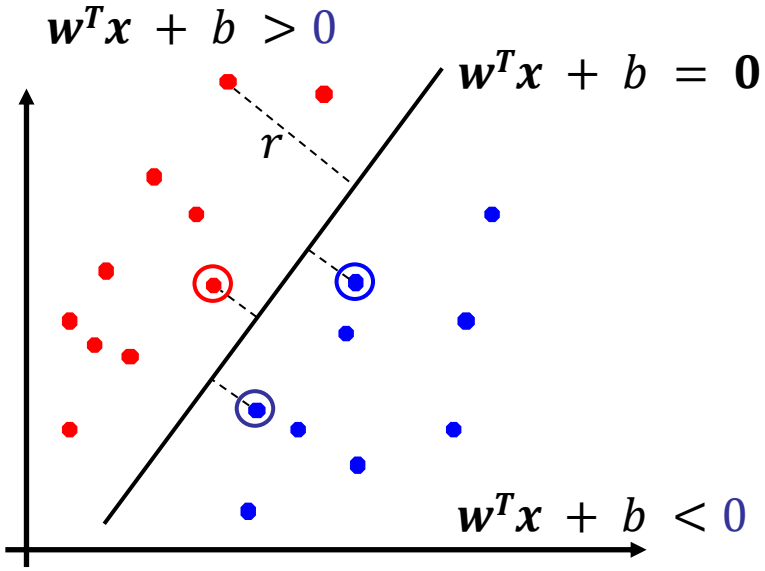
Logistic Regression:

$$p(y = +1 | \boldsymbol{x}_i, \boldsymbol{w}, b) = \operatorname{sigmoid}(\boldsymbol{w}^T\boldsymbol{x} + b)$$

$$L(p, y) = -y \log p - (1 - y) \log(1 - p)$$

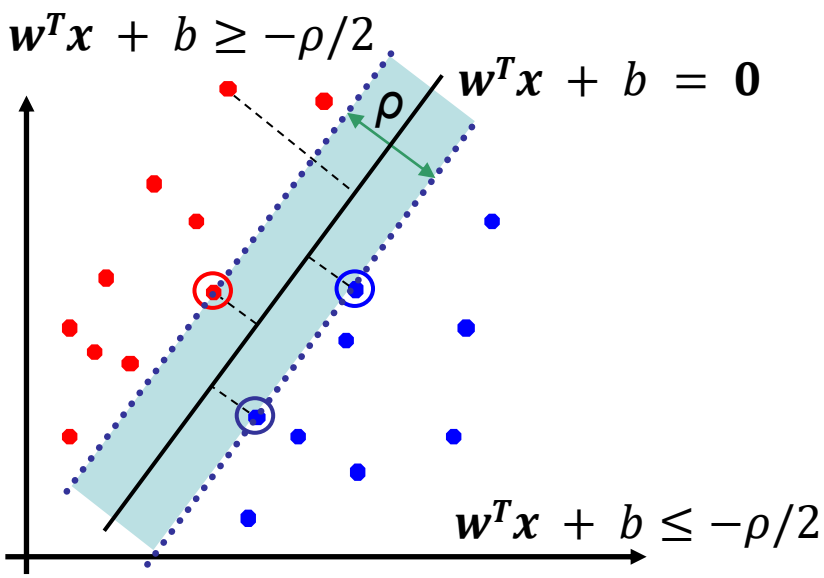
Classification Margin

- Distance from example $\mathbf{x}_i$ to the separator is $r = \dfrac{\boldsymbol{w}^T \boldsymbol{x}_i + b}{\|\boldsymbol{w}\|}$
- Examples closest to the hyperplane are ***support vectors***.



Classification Margin

- Examples closest to the hyperplane are ***support vectors***.
- ***Margin*** ρ of the separator is the distance between support vectors.



$$\boldsymbol{w}^T\boldsymbol{x}_i + b \geq \rho/2 \quad \text{if } y_i = 1$$

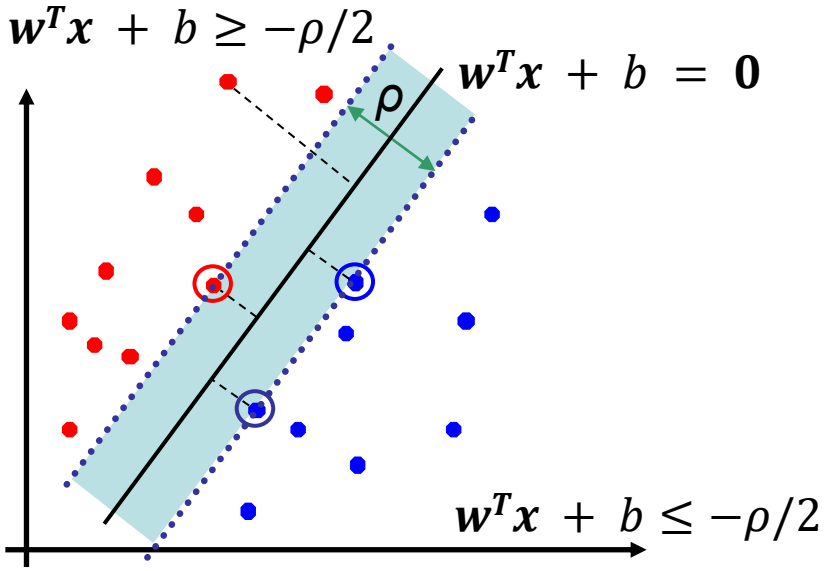
$$\boldsymbol{w}^T\boldsymbol{x}_i + b \leq -\rho/2 \quad \text{if } y_i = -1$$

$$\Updownarrow$$

$$y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq \rho/2$$

Maximum Margin Classification

- Examples closest to the hyperplane are ***support vectors***.
- ***Margin*** ρ of the separator is the distance between support vectors.



$$\boldsymbol{w}^T\boldsymbol{x}_i + b \geq \rho/2 \quad \text{if } y_i = 1$$

$$\boldsymbol{w}^T\boldsymbol{x}_i + b \leq -\rho/2 \quad \text{if } y_i = -1$$

$$\Updownarrow$$

$$y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq \rho/2$$

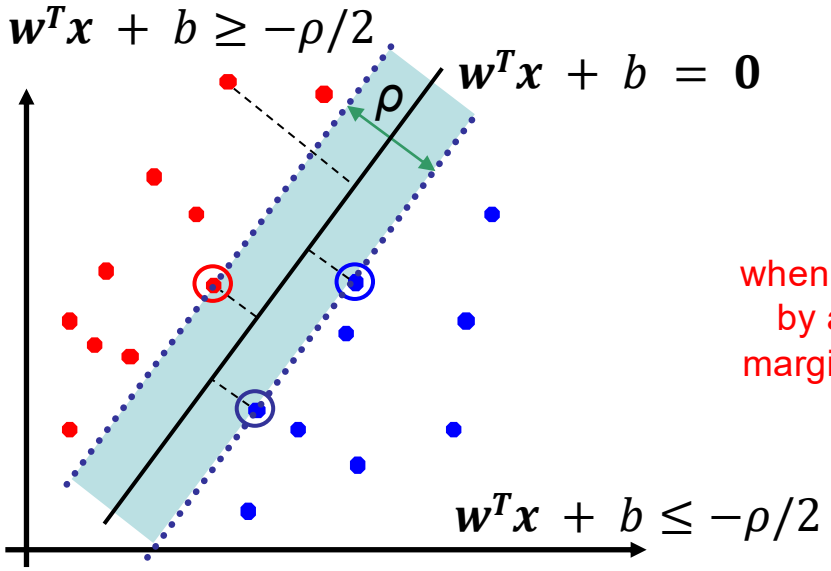
$$\Downarrow$$

Basic Idea: Maximize ρ

Maximum Margin Classification

- Examples closest to the hyperplane are ***support vectors***.
- ***Margin*** ρ of the separator is the distance between support vectors.

$$y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq \rho/2$$



support vectors

$$\frac{\rho}{2} = \frac{\boldsymbol{w}^T\boldsymbol{x}_s + b}{\|\boldsymbol{w}\|}$$

when (w, b) scaled by a factor c, the margin unchanged

$$= \frac{c\boldsymbol{w}^T\boldsymbol{x}_s + cb}{\|c\boldsymbol{w}\|}$$

$$= \frac{1}{\|\boldsymbol{w}\|}$$

SVM: simplified objective

- Then we can formulate the *quadratic optimization problem:*

Find $\boldsymbol{w}$ and b such that $\rho = \frac{2}{||\mathbf{w}||}$ is maximized

and for all $(\boldsymbol{x}_i, y_i), i = 1,2,\cdots,n$: $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1$

- Which can be reformulated as:

Find $\boldsymbol{w}$ and b such that $\phi(\boldsymbol{w}) = \left|\left|\boldsymbol{w}\right|\right|^2 = \boldsymbol{w}^T\boldsymbol{w}$ is minimized

and for all $(\boldsymbol{x}_i, y_i), i = 1,2,\cdots,n$: $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1$

Solving the Optimization Problem

Find $\boldsymbol{w}$ and b such that $\phi(\boldsymbol{w}) = ||\boldsymbol{w}||^2 = \boldsymbol{w}^T\boldsymbol{w}$ is minimized

and for all $(\boldsymbol{x}_i, y_i), i = 1,2,\cdots,n$: $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1$

- Gradient descend?
- What about the constraint?

Lagrangian Function & The Dual Problem

Using a Lagrangian

- A general convex optimization problem

$$\begin{aligned} &\text{Minimize}_w && f(w) \\ &s.t. && g(w) \leq 0 \end{aligned}$$

- Lagrangian function: $L(w, \lambda) = f(w) + \lambda^T g(w)$

- The original problem can be written as

$$\begin{aligned} &\min_w \max_\lambda && L(w, \lambda) = f(w) + \lambda^T g(w) \\ &s.t. && \lambda \geq 0 \end{aligned}$$

http://cs229.stanford.edu/notes2020fall/notes2020fall/cs229-notes3.pdf
Page 17~20

Using a Lagrangian

$$\begin{aligned} &\text{Minimize}_w && f(w) \\ &s.t. && g(w) \le 0 \end{aligned}$$

$\Leftrightarrow$

Why?

$$\begin{aligned} &\min_w \max_\lambda && L(w,\lambda) = f(w) + \lambda^T g(w) \\ &s.t. && \lambda \ge 0 \end{aligned}$$

$$L(w,\lambda) = \begin{cases} f(w), & \text{if } w \text{ satisfies all the constraints} \\ +\infty, & \text{if } w \text{ does not satisfy all the constraints} \end{cases}$$

So minimizing $f(w)$ is the same as minimizing $L(w,\lambda)$

http://cs229.stanford.edu/notes2020fall/notes2020fall/cs229-notes3.pdf
Page 17~20

Using a Lagrangian

$$\begin{aligned} &\text{Minimize}_w && f(w) \\ &s.t. && g(w) \leq 0 \end{aligned}$$

$\Leftrightarrow$

Why?

$$\begin{aligned} &\min_w \max_\lambda && L(w,\lambda) = f(w) + \lambda^T g(w) \\ &s.t. && \lambda \geq 0 \end{aligned}$$

Case 1: The Constraint is Satisfied ($g(w) \leq 0$)

If w is a value that satisfies the original constraint, then $g(w)$ is a negative number or zero. In this case, to maximize $L(w, \lambda) = f(w) + \lambda^T g(w)$ with respect to λ (where $\lambda \geq 0$), we need to make the term $\lambda^T g(w)$ as large as possible.

- Since $g(w) \leq 0$ and we must have $\lambda \geq 0$, the product $\lambda^T g(w)$ will always be less than or equal to zero.
- To maximize this term, the best we can do is make it zero by setting $\lambda = 0$.
- Therefore, when the constraint is satisfied, the result of the maximization is just $f(w)$:

$$\max_{\lambda \geq 0} L(w, \lambda) = f(w) + 0 = f(w)$$

Using a Lagrangian

$$\begin{aligned} &\text{Minimize}_w && f(w) \\ &s.t. && g(w) \leq 0 \end{aligned}$$

$\Leftrightarrow$

Why?

$$\begin{aligned} &\min_w \max_\lambda && L(w,\lambda) = f(w) + \lambda^T g(w) \\ &s.t. && \lambda \geq 0 \end{aligned}$$

Case 2: The Constraint is Violated ($g(w) > 0$)

If w is a value that violates the original constraint, then $g(w)$ is a positive number. Now, when we try to maximize $L(w, \lambda) = f(w) + \lambda^T g(w)$, the situation changes:

- Since $g(w) > 0$ and λ can be any non-negative number, we can make the term $\lambda^T g(w)$ infinitely large by letting λ go to infinity.
- Therefore, if the constraint is violated, the result of the maximization is infinity:

$$\max_{\lambda \geq 0} L(w, \lambda) = f(w) + \infty = \infty$$

Using a Lagrangian

$$\begin{aligned} &\text{Minimize}_w && f(w) \\ &s.t. && g(w) \leq 0 \end{aligned}$$

$\Leftrightarrow$

Why?

$$\begin{aligned} &\min_w \max_\lambda && L(w,\lambda) = f(w) + \lambda^T g(w) \\ &s.t. && \lambda \geq 0 \end{aligned}$$

Putting It All Together

Now, let's look at the full "min-max" problem: $\min_w \left(\max_{\lambda \geq 0} L(w, \lambda)\right)$.

Based on our two cases, this is equivalent to:

$$\min_w \begin{cases} f(w) & \text{if } g(w) \leq 0 \\ \infty & \text{if } g(w) > 0 \end{cases}$$

When we are trying to find the *minimum* value, the optimizer will naturally discard any value of w that results in ∞. It will only consider the values of w where the result is $f(w)$, which are precisely the values that satisfy the constraint $g(w) \leq 0$.

The Dual Problem

$$\min_{w} \max_{\lambda} \quad L(w,\lambda) = f(w) + \lambda^T g(w) \qquad \geq \qquad \max_{\lambda} \min_{w} \quad L(w,\lambda) = f(w) + \lambda^T g(w)$$
$$s.t. \qquad \lambda \geq 0$$

Why?

When do we have?

$=$

w^*, λ^* satisfy Karush-Kuhn-Tucker (KKT) conditions:

$$\frac{\partial L}{\partial w^*} = 0 \qquad \lambda^{*T} g(w^*) = 0$$
$$g(w^*) \leq 0 \qquad \lambda^* \geq 0$$

Dual complementarity

Primary constraint

Dual constraint

http://cs229.stanford.edu/notes2020fall/notes2020fall/cs229-notes3.pdf
Page 17~20

SVM : Optimization

$$\min_{w} \max_{\lambda} \quad L(w,\lambda) = f(w) + \lambda^T g(w)$$
$$s.t. \qquad \lambda \geq 0$$

$\Updownarrow$ When Karush-Kuhn-Tucker (KKT) conditions hold

$$\max_{\lambda} \min_{w} \quad L(w,\lambda) = f(w) + \lambda^T g(w)$$

$$\frac{\partial L}{\partial w} = 0 \qquad \lambda^T g(w) = 0$$

$$g(w) \leq 0 \qquad \lambda \geq 0$$

$$\min_{w,b} \max_{\lambda} \quad \frac{1}{2} w^T w + \sum_{i=1}^{n} \lambda_i (1 - y_i (w^T x_i + b))$$
$$s.t. \qquad \lambda \geq 0$$

$$\Updownarrow$$

$$\max_{\lambda} \underbrace{\min_{w,b} \frac{1}{2} w^T w + \sum_{i=1}^{n} \lambda_i (1 - y_i (w^T x_i + b))}$$

$$\frac{\partial L(w,b,\lambda)}{\partial w} = 0 \implies w = \sum_{i}^{n} \lambda_i y_i x_i$$

$$\frac{\partial L(w,b,\lambda)}{\partial b} = 0 \implies 0 = \sum_{i}^{n} \lambda_i y_i$$

http://cs229.stanford.edu/notes2020fall/notes2020fall/cs229-notes3.pdf
Page 17~20

SVM： Optimization

$$\max_{\lambda} \quad \theta(\lambda) = \sum_i \lambda_i - \frac{1}{2}\sum_i \sum_j \lambda_i \lambda_j y_i y_j x_i^T x_j$$

$$\text{s.t.} \quad \lambda \geq 0, \quad \sum_i \lambda_i y_i = 0$$

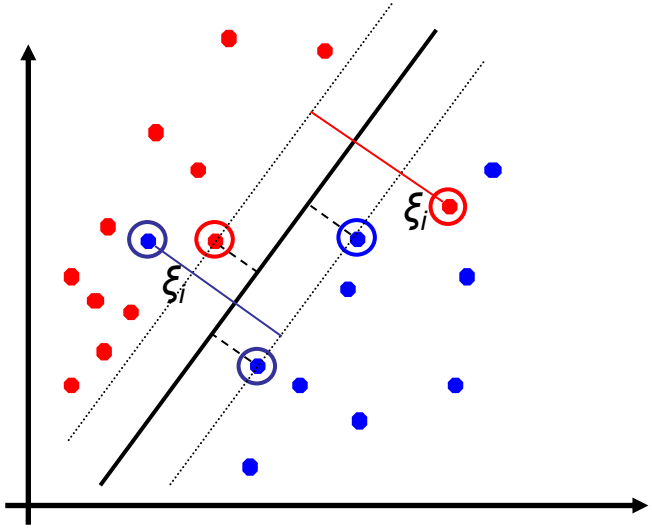
The SMO (sequential minimal optimization) algorithm
http://cs229.stanford.edu/materials/smo.pdf

$$w^* = \sum_i \lambda_i^* y_i x_i$$

$$b^* = -\frac{\max_{i:y_i=-1} w^{*T} x_i + \min_{i:y_i=1} w^{*T} x_i}{2}$$

Soft Margin Classification

- What if the training set is not linearly separable?
- *Slack variables* ξ_i can be added to allow misclassification of difficult or noisy examples, resulting margin called *soft*.



Soft Margin Classification Mathematically

- The old formulation:

  Find $\boldsymbol{w}$ and b such that $\phi(\boldsymbol{w}) = \boldsymbol{w}^T\boldsymbol{w}$ is minimized

  and for all ($\mathbf{x}_i$, y_i), i=1..n : $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1$

- Modified formulation incorporates slack variables:

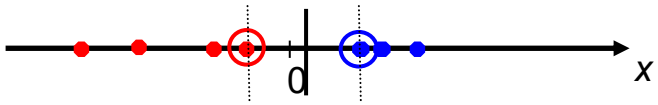
  Find $\mathbf{w}$ and b such that $\phi(\mathbf{w}) = \mathbf{w}^{\mathrm{T}}\mathbf{w} + \mathrm{C}\sum_i \xi_i$ is minimized

  and for all ($\mathbf{x}_i$, y_i), i=1..n : $\mathrm{y_i}\left(\mathbf{w}^{\mathrm{T}}\mathbf{x}_{\mathrm{i}} + \mathrm{b}\right) \geq 1 - \xi_i, \quad \xi_i \geq 0$

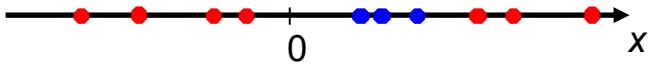
- Parameter C can be viewed as a way to **control overfitting**: it "trades off" the relative importance of maximizing the margin and fitting the training data.

Non-linear SVMs

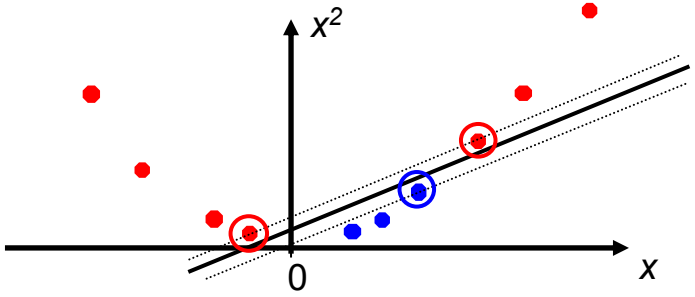
- Datasets that are linearly separable with some noise work out great:



- But what are we going to do if the dataset is just too hard?

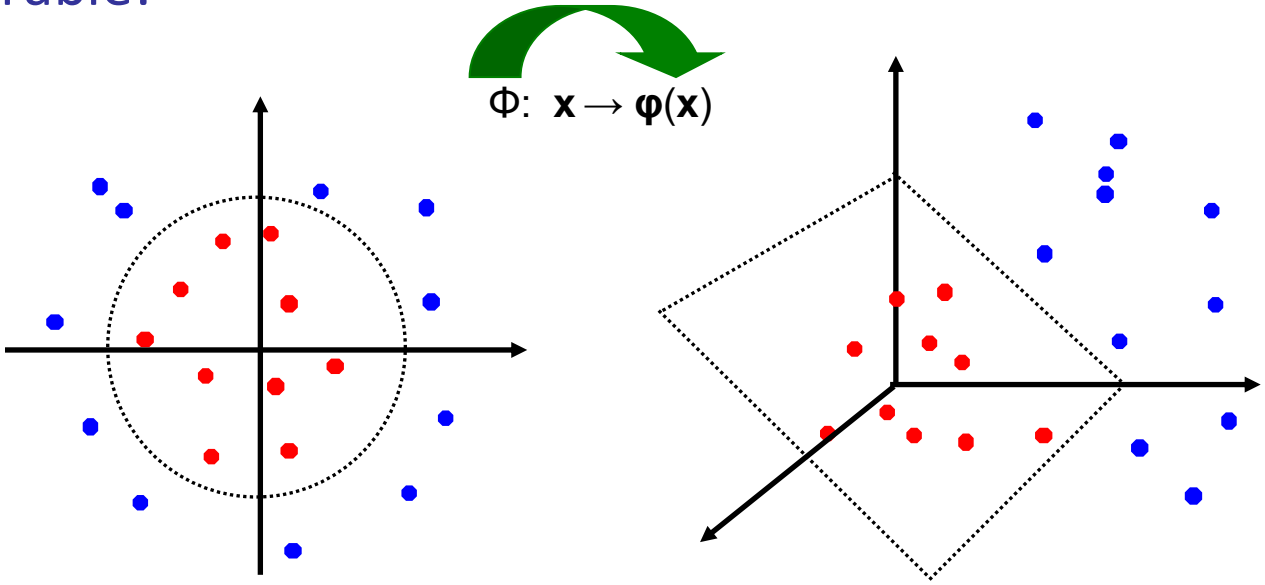


- How about… mapping data to a higher-dimensional space:

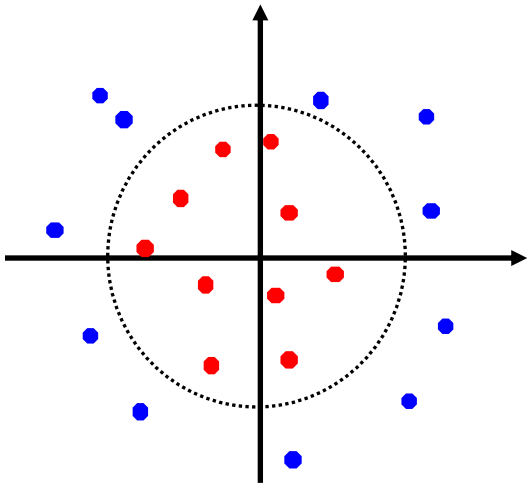


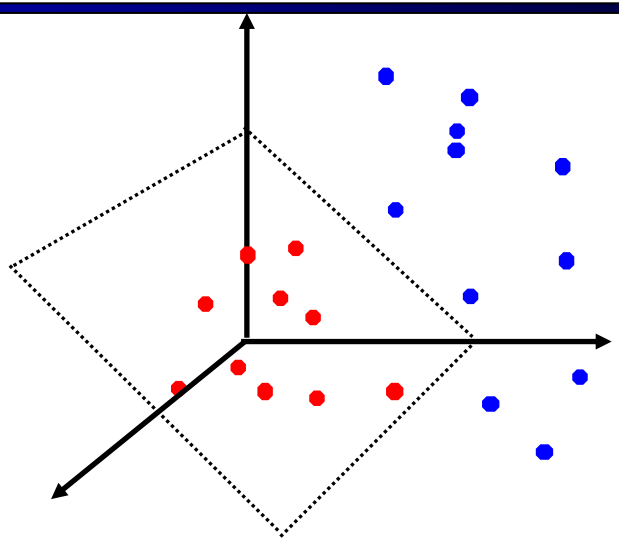
Non-linear SVMs: Feature spaces

- General idea: the original feature space can always be mapped to some higher-dimensional feature space where the training set is separable:



Non-linear SVMs: Feature spaces





(x_1, x_2) ⇒ $(z_1, z_2, z_3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$

⇓

$w_1x_1^2 + w_2\sqrt{2}x_1x_2 + w_3x_2^2 + b = 0$ ⇐ $w^Tz + b = 0$

The Kernel Trick

- Recall SVM Dual optimization problem

$$\max_{\lambda} \quad \theta(\lambda) = \sum_i \lambda_i - \frac{1}{2}\sum_i \sum_j \lambda_i \lambda_j y_i y_j x_i^T x_j$$

$$\text{s.t.} \quad \lambda \geq 0, \quad \sum_i \lambda_i y_i = 0$$

- The data points only appear as inner product

- Define the kernel function K by

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

An Example for ϕ(.) and K(.,.)

- Suppose ϕ(.) is given as follows

$$\phi(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

- An inner product in the feature space is

$$\langle \phi(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}), \phi(\begin{bmatrix} y_1 \\ y_2 \end{bmatrix}) \rangle = (1 + x_1y_1 + x_2y_2)^2$$

- So, if we define the kernel function as follows, there is no need to carry out ϕ(.) explicitly

$$K(\mathbf{x}, \mathbf{y}) = (1 + x_1y_1 + x_2y_2)^2$$

- This use of kernel function to avoid carrying out ϕ(.) explicitly is known as the kernel trick

More Examples for $\phi(.)$ and K(.,.)

- Let's define a kernel function $K(\boldsymbol{x}, \boldsymbol{y}) = (\boldsymbol{x}^T \boldsymbol{y})^2$
- It can be decomposed to

$$K(\boldsymbol{x}, \boldsymbol{y}) = (\boldsymbol{x}^T \boldsymbol{y})^2 = \left(\sum_{i=1}^{d} x_i y_i\right)\left(\sum_{j=1}^{d} x_j y_j\right) = \sum_{i=1}^{d}\sum_{j=1}^{d} (x_i x_j)(y_i y_j)$$

- We can find the feature mapping function $\phi(x) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ x_1 x_3 \\ x_2 x_1 \\ x_2 x_2 \\ x_2 x_3 \\ x_3 x_1 \\ x_3 x_2 \\ x_3 x_3 \end{bmatrix}$

What Functions are Kernels?

- For some functions $K(x_i, x_j)$ checking that $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ can be cumbersome.
- Mercer's theorem:

Every semi-positive definite symmetric function is a kernel, i.e.,

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Theorem (Mercer). Let $K : \mathbb{R}^d \times \mathbb{R}^d \mapsto \mathbb{R}$ be given. Then for K to be a valid (Mercer) kernel, it is necessary and sufficient that for any $\{x^{(1)}, \ldots, x^{(n)}\}$, $(n < \infty)$, the corresponding kernel matrix is symmetric positive semi-definite.

What Functions are Kernels?

- Mercer's theorem:

Every semi-positive definite symmetric function is a kernel, i.e.,

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

- Semi-positive definite symmetric functions correspond to a semi-positive definite symmetric Gram matrix:

K=

$K(\boldsymbol{x}_1, \boldsymbol{x}_1)$	$K(\boldsymbol{x}_1, \boldsymbol{x}_2)$	$K(\boldsymbol{x}_1, \boldsymbol{x}_3)$	…	$K(\boldsymbol{x}_1, \boldsymbol{x}_n)$
$K(\boldsymbol{x}_2, \boldsymbol{x}_1)$	$K(\boldsymbol{x}_2, \boldsymbol{x}_2)$	$K(\boldsymbol{x}_2, \boldsymbol{x}_3)$		$K(\boldsymbol{x}_2, \boldsymbol{x}_n)$
…	…	…	…	…
$K(\boldsymbol{x}_n, \boldsymbol{x}_1)$	$K(\boldsymbol{x}_n, \boldsymbol{x}_2)$	$K(\boldsymbol{x}_n, \boldsymbol{x}_3)$	…	$K(\boldsymbol{x}_n, \boldsymbol{x}_n)$

Examples of Kernel Functions

- Linear: $K(x_i, x_j) = x_i^T x_j$
  - Mapping Φ: **x** → **ϕ**(**x**), where **ϕ**(**x**) is **x** itself
- Polynomial of power *p*: $K(x_i, x_j) = (1 + x_i^T x_j)^p$
- Gaussian (radial-basis function): $K(x_i, x_j) = \exp(-\frac{||x_i - x_j||^2}{2\sigma^2})$

Non-linear SVMs Mathematically

- Dual problem formulation:

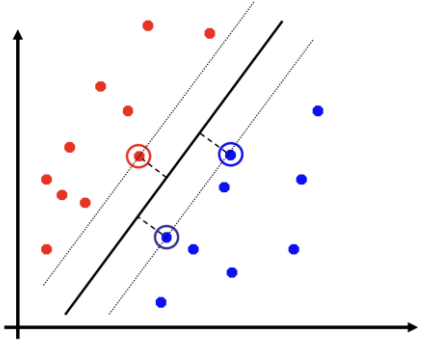
$$\max_{\lambda} \quad \theta(\lambda) = \sum_i \lambda_i - \frac{1}{2}\sum_i \sum_j \lambda_i \lambda_j y_i y_j K(x_i, x_j)$$

$$\text{s.t.} \quad \lambda \geq 0, \quad \sum_i \lambda_i y_i = 0$$

- Optimization techniques for finding λ_i's remain the same!

Recap: SVM

- Maximum Margin Classification



- The Dual Problem of maximizing margin

$$\max_{\lambda} \quad \theta(\lambda) = \sum_i \lambda_i - \frac{1}{2}\sum_i \sum_j \lambda_i \lambda_j y_i y_j x_i^T x_j$$

$$\text{s.t.} \quad \lambda \geq 0, \quad \sum_i \lambda_i y_i = 0$$

- Soft Margin Classification

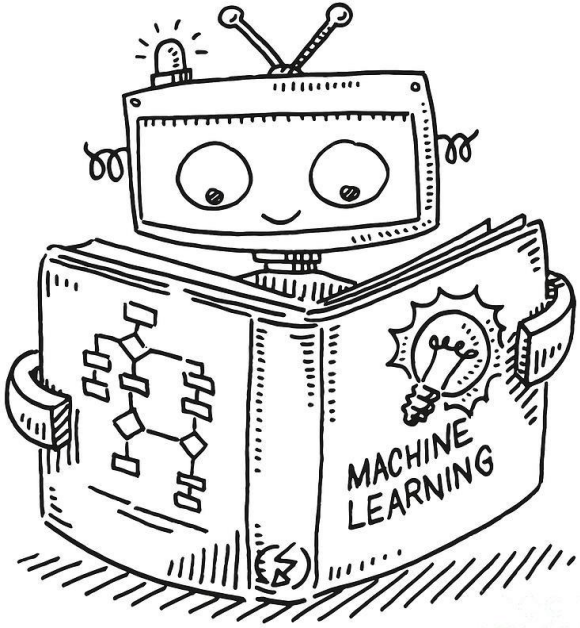
- The Kernel Trick

$$\max_{\lambda} \quad \theta(\lambda) = \sum_i \lambda_i - \frac{1}{2}\sum_i \sum_j \lambda_i \lambda_j y_i y_j K(x_i, x_j)$$

$$\text{s.t.} \quad \lambda \geq 0, \quad \sum_i \lambda_i y_i = 0$$

Today

- Classification: Support Vector Machine
- **Classification: Decision Tree**
- Discussion



Decision Tree

- What makes a loan risky?

Loan Applicatio n

Credit
★★★★

Income
★★★

Term
★★★★★

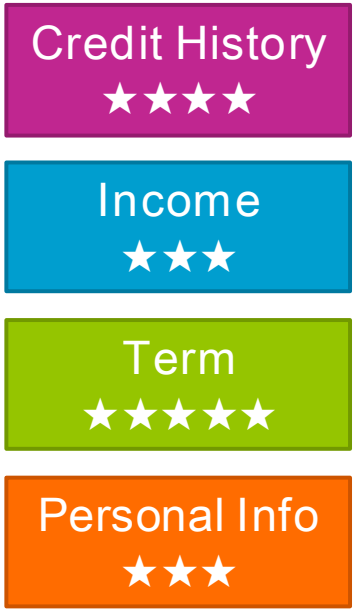
Personal Info
★★★

Decision Tree

- Credit history explained

Did I pay previous loans on time?

Example: excellent, good, or fair

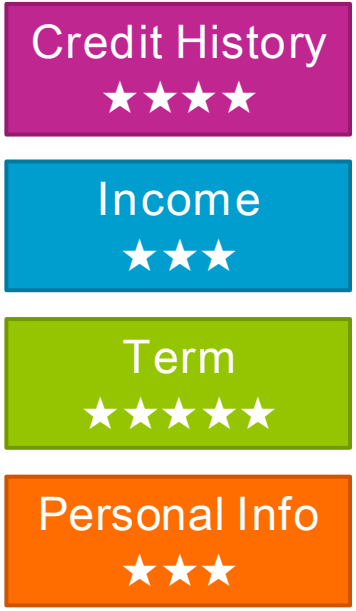


Decision Tree

- Credit history explained

What's my income?

Example
$80K per year

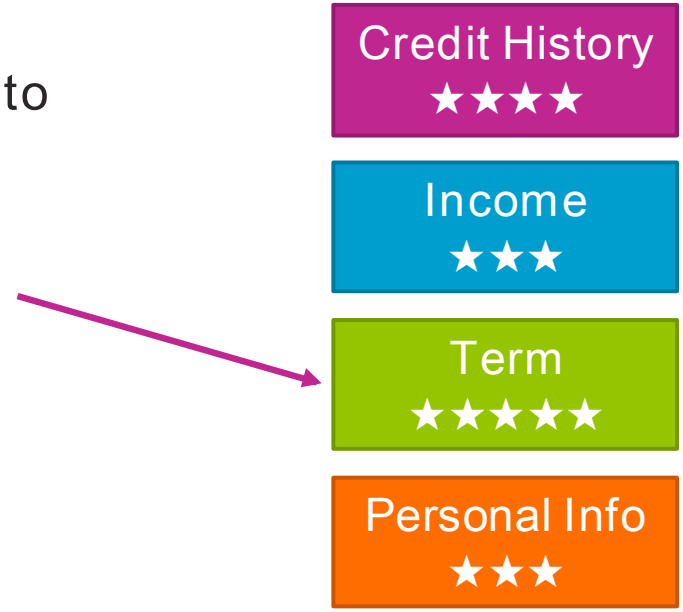


Decision Tree

- Credit history explained

How soon do I need to pay the loan?

Example: 3 years, 5 years,…

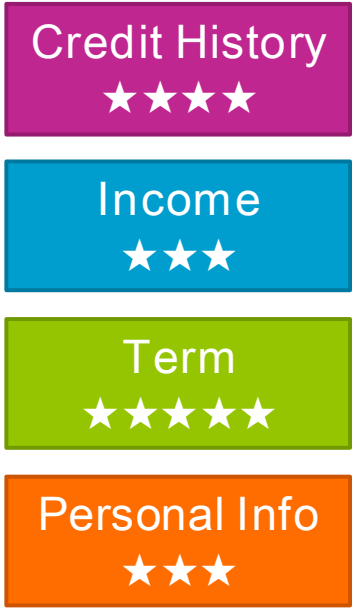


Decision Tree

- Credit history explained

Age, reason for the loan, marital status,…

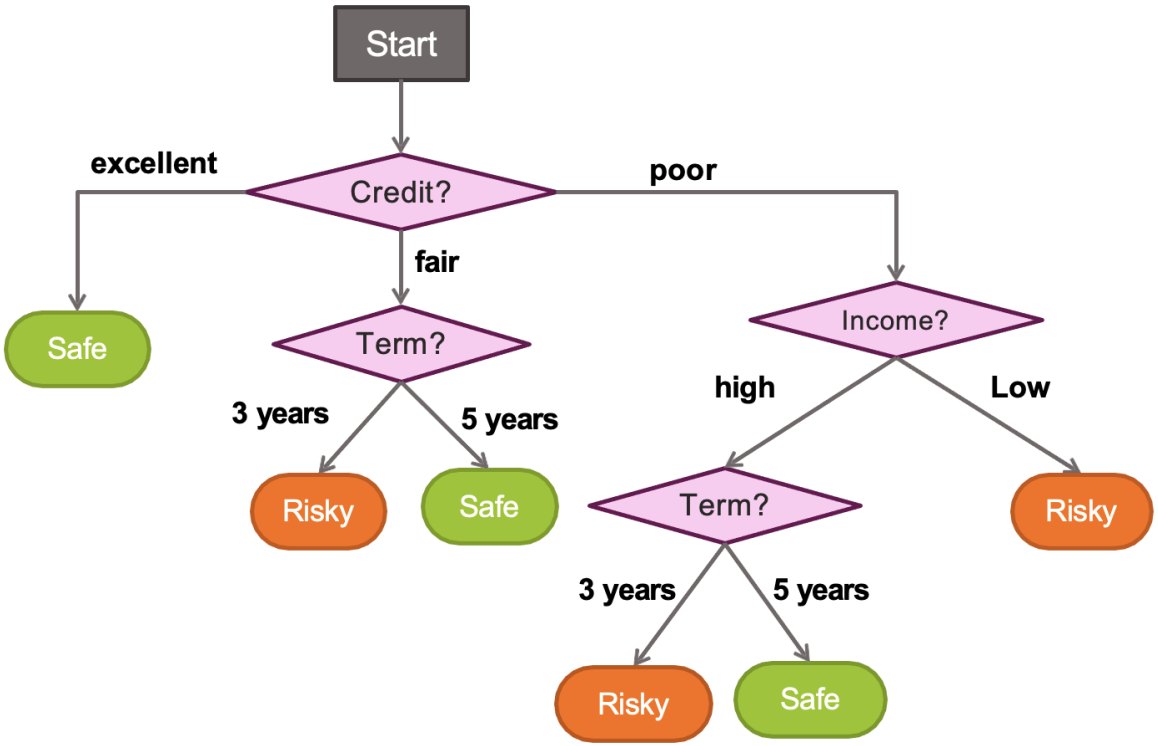
Example: Home loan for a married couple



Decision Tree

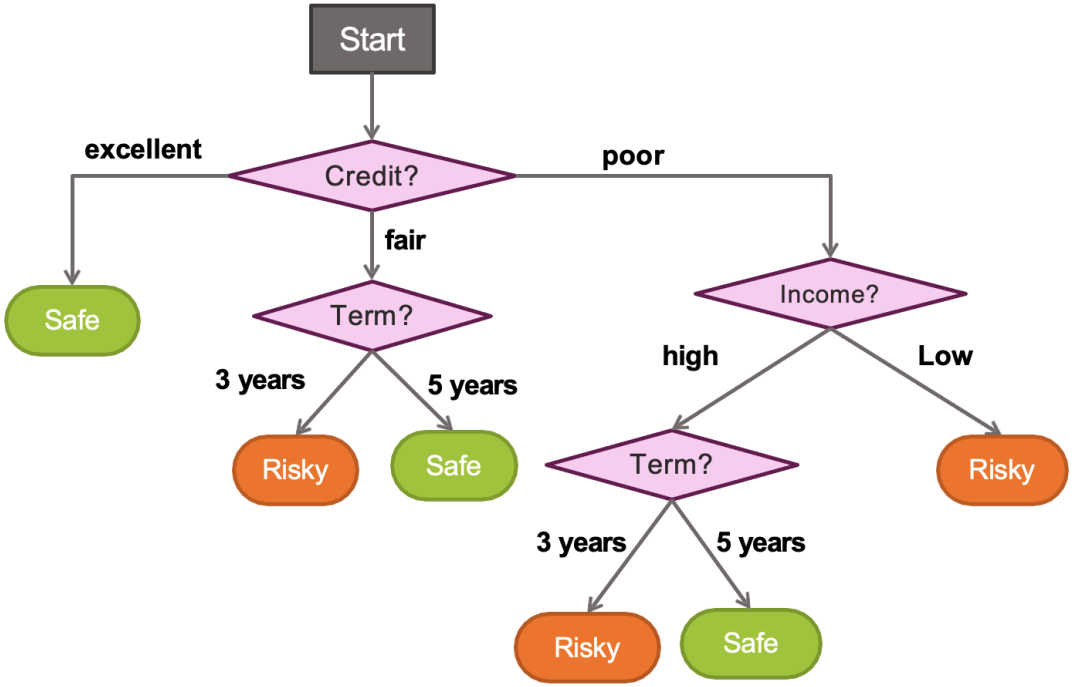
- Intelligent application

Decision Tree



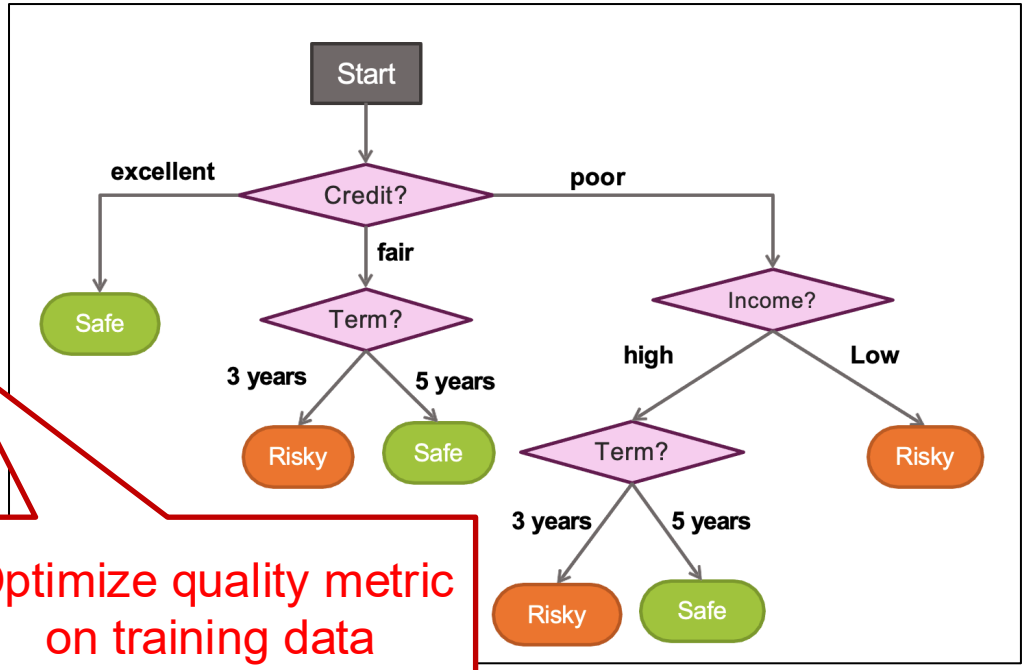
Scoring a loan application

- x_i = (Credit = poor, Income = high, Term = 5 years)



Learn decision tree from data

Credit	Term	Income	y
excellent	3 yrs	high	safe
fair	5 yrs	low	risky
fair	3 yrs	high	safe
poor	5 yrs	high	risky
excellent	3 yrs	low	risky
fair	5 yrs	low	safe
poor	3 yrs	high	risky
poor	5 yrs	low	safe
fair	3 yrs	high	safe



Optimize quality metric on training data

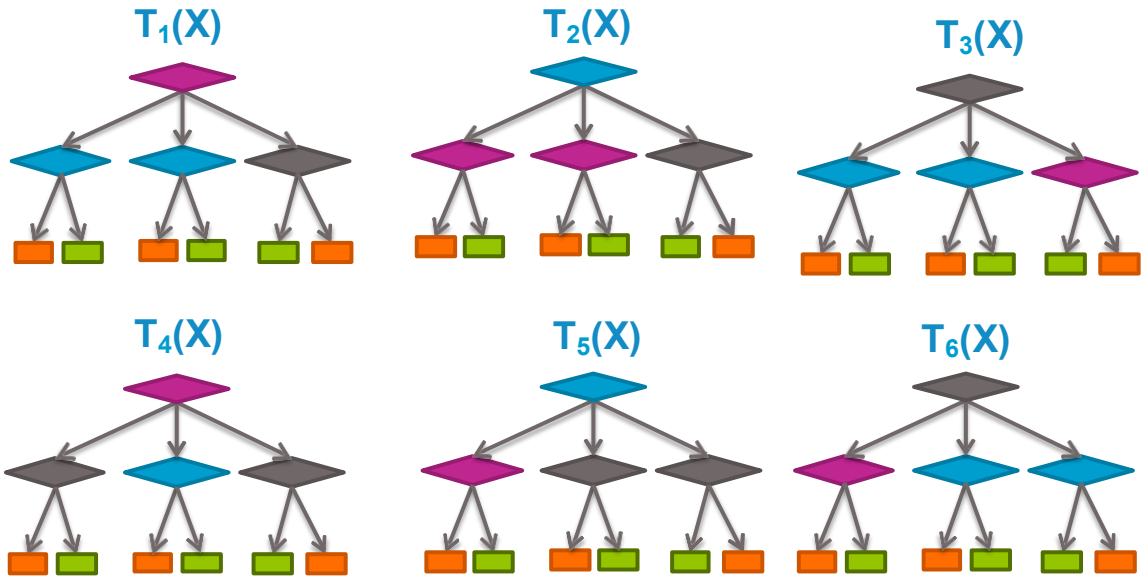
Quality metric: Classification error

- Error measures fraction of mistakes

$$Error = \frac{\#\text{incorrect predictions}}{\#examples}$$

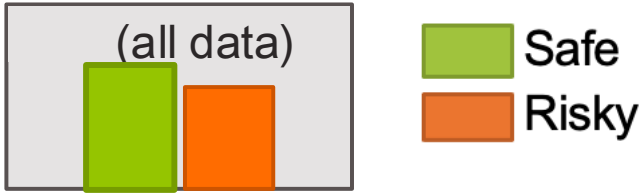
How do we find the best tree?

- Exponentially large number of possible trees makes decision tree learning hard! *(NP-hard problem)*



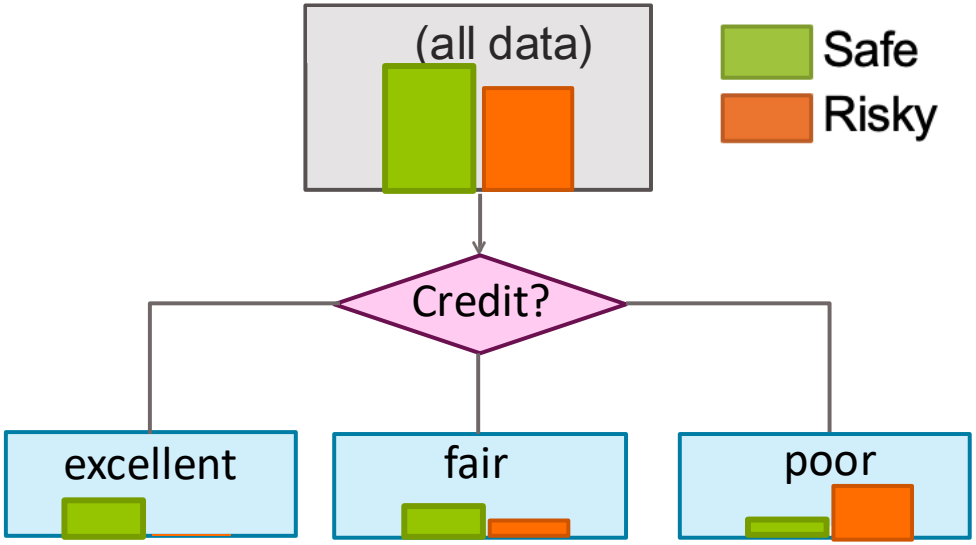
Greedy Decision Tree Learning

- Step 1: Start with an empty tree



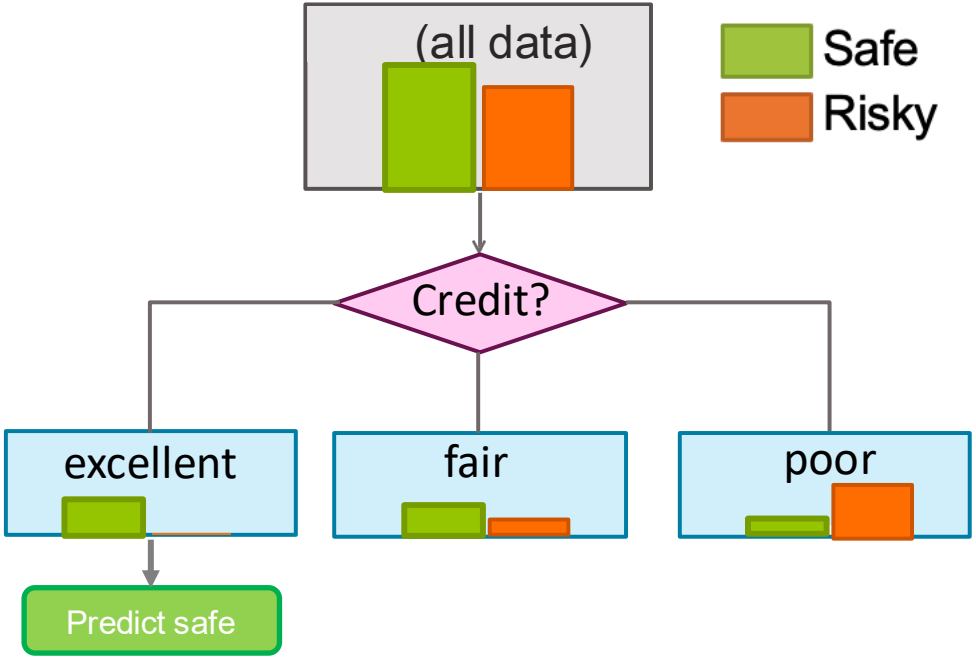
Greedy Decision Tree Learning

- Step 2: Split on a feature



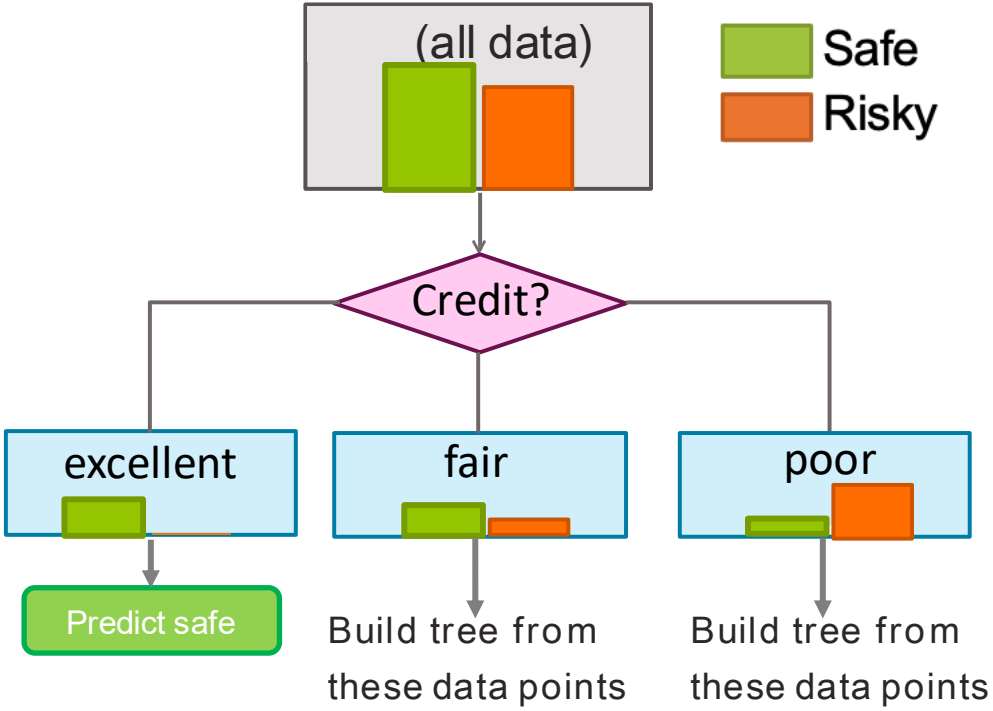
Greedy Decision Tree Learning

- Step 3: Making predictions



Greedy Decision Tree Learning

- Step 4: Recursion



Greedy Decision Tree Learning

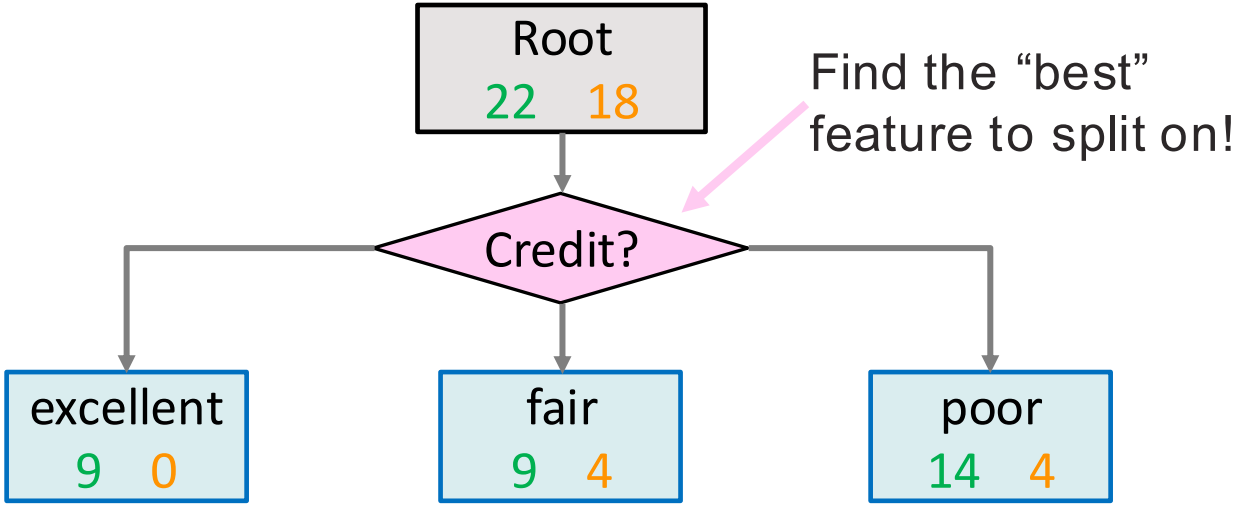
- Step 1: Start with an empty tree
- Step 2: Select a feature to split data
- For each split of the tree:
  - Step 3: If nothing more to do, make predictions
  - Step 4: Otherwise, go to Step 2 & continue (recurse) on this split

Problem 1: Feature split selection

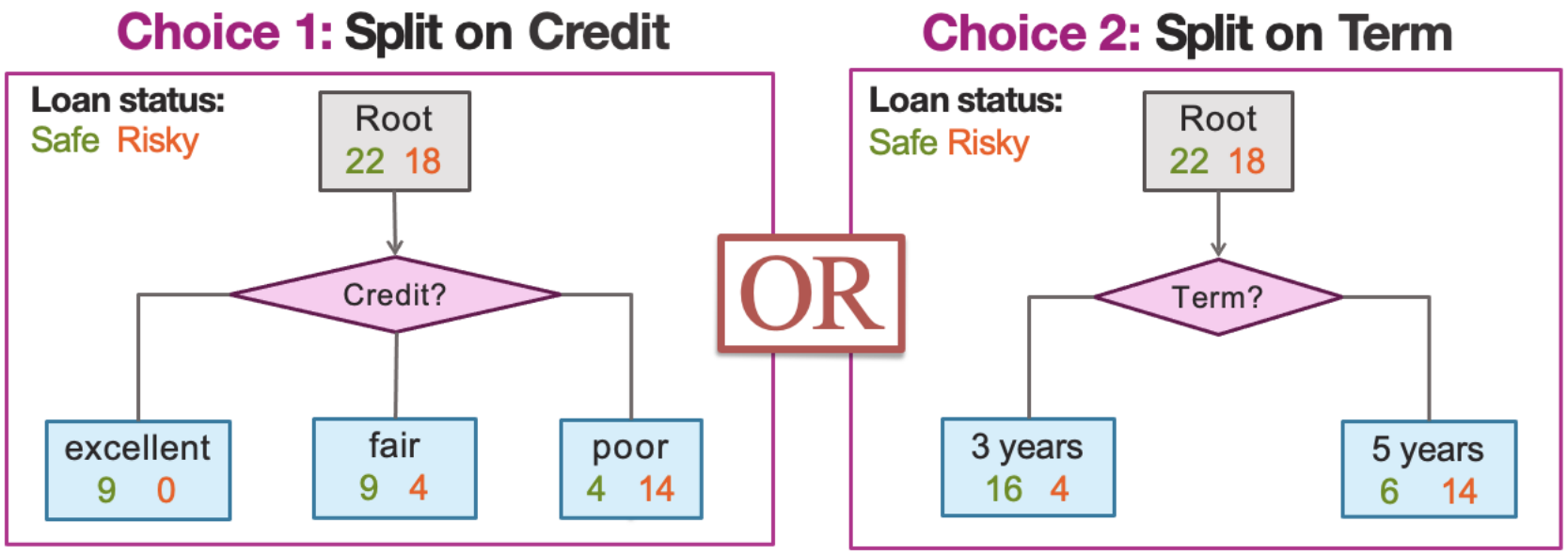
Problem 2: Stopping condition

Recursion

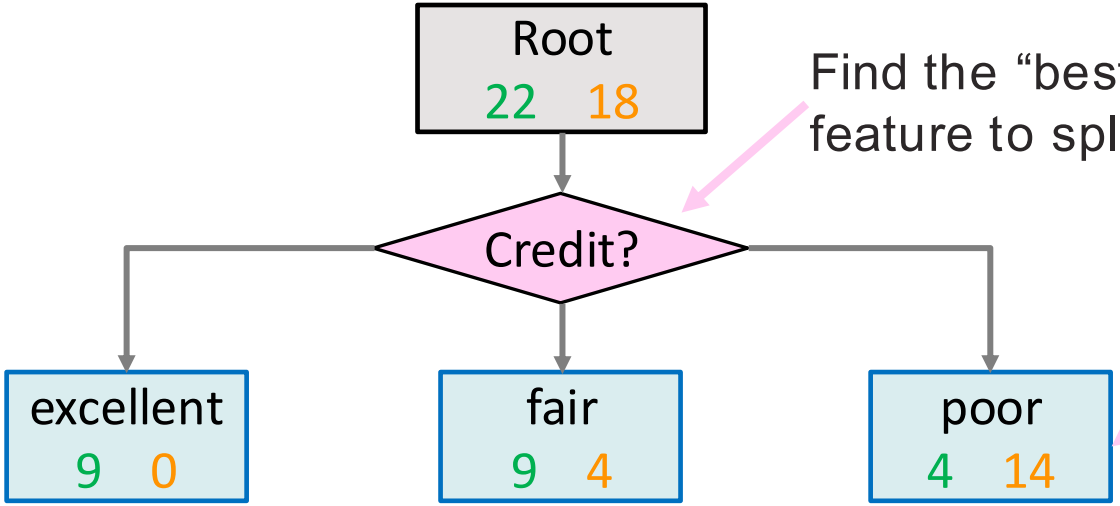
Feature split selection



Feature split selection



How to measure effectiveness of a split

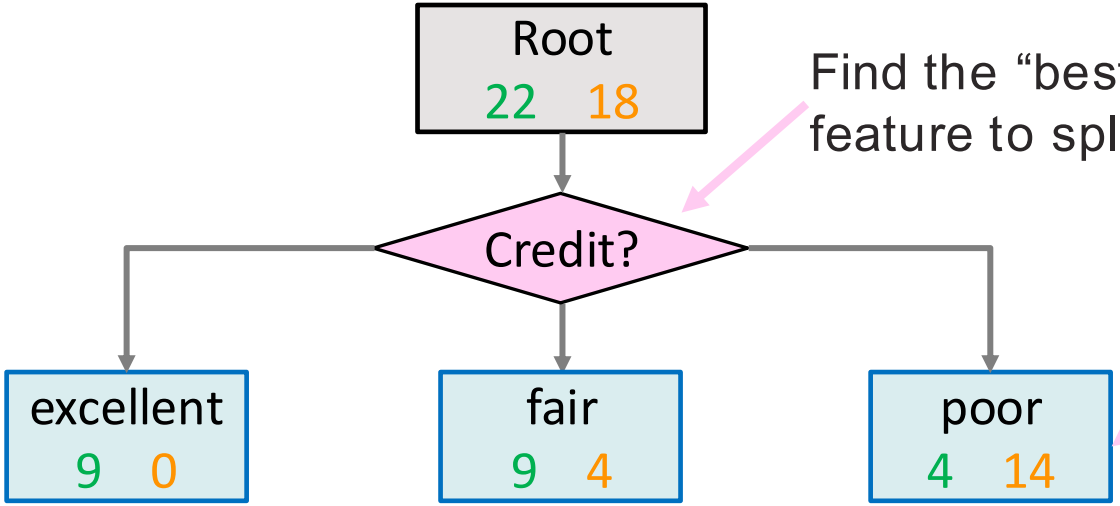


Find the "best" feature to split on!

Idea: Calculate classification error of this decision stump

$$Error = \frac{\#incorrect\ predictions}{\#examples}$$

Calculating classification error



Find the "best" feature to split on!

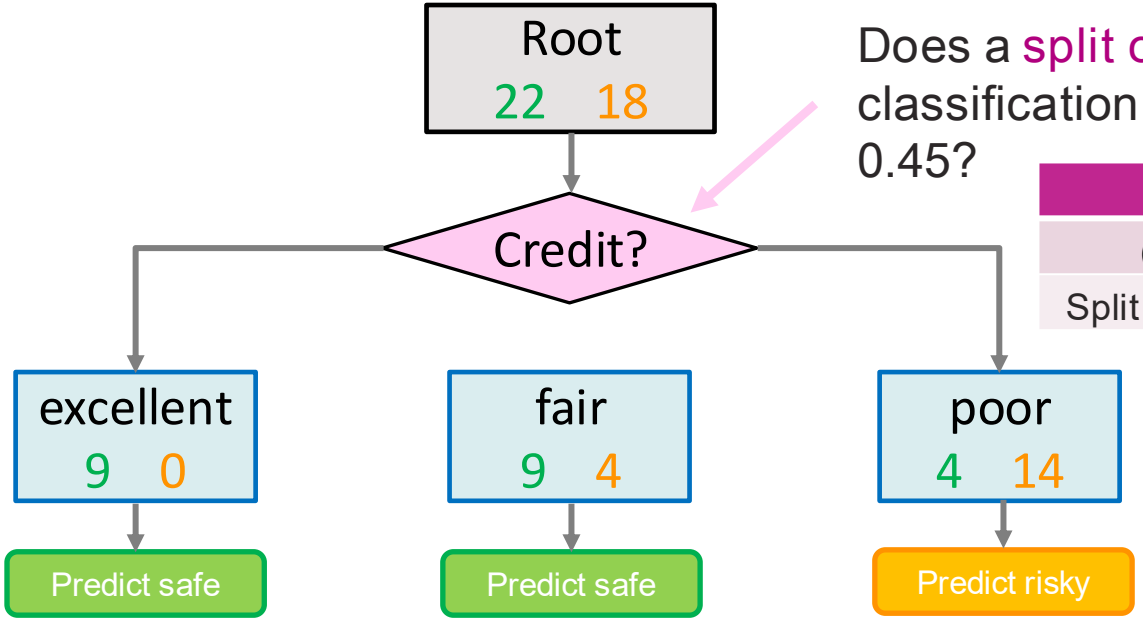
Idea: Calculate classification error of this decision stump

Step 1: ŷ = class of majority of data in node

Step 2: Calculate classification error of predicting ŷ for this data

$$Error = \frac{\#incorrect\ predictions}{\#examples}$$

Choice 1: Split on credit history?



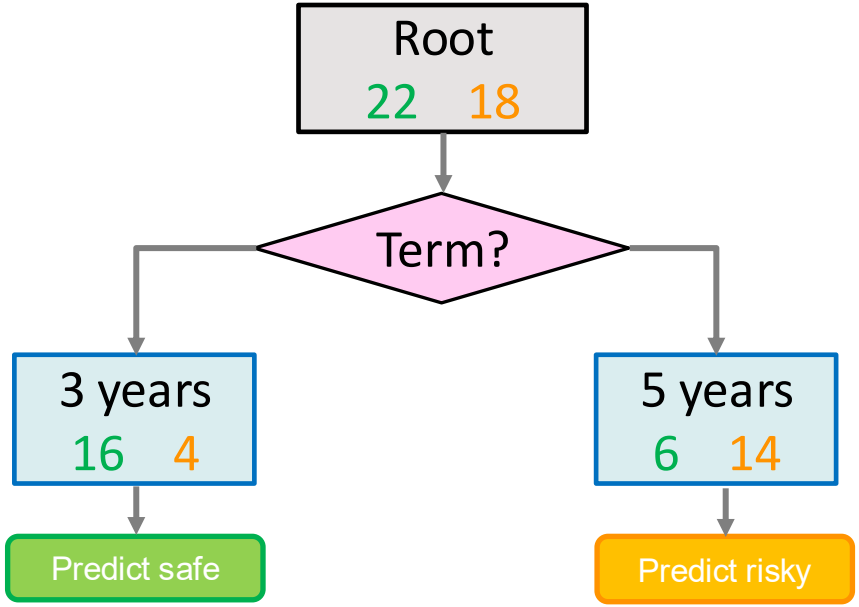
Does a split on Credit reduce classification error below 0.45?

Tree	Classification error
(root)	0.45
Split on credit	0.2

Step 1: $\hat{y}$ = class of majority of data in node

Step 2: Calculate classification error of predicting $\hat{y}$ for this data

Choice 2: Split on Term?

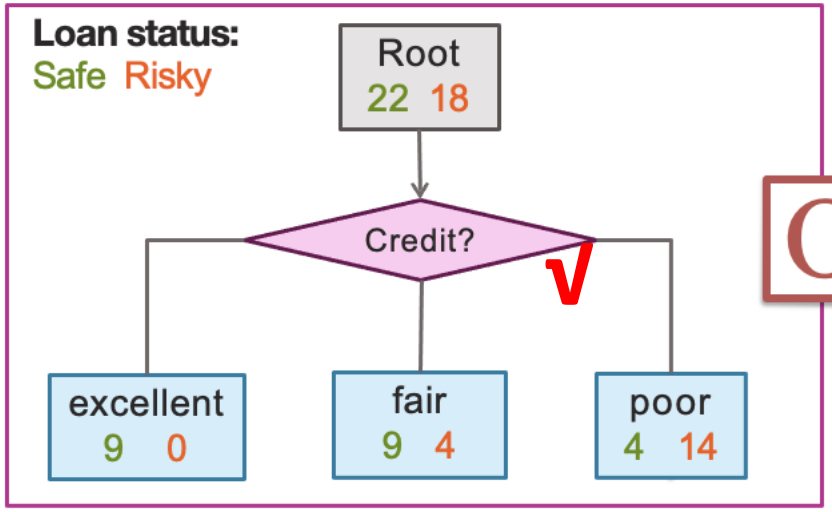


Tree	Classification error
(root)	0.45
Split on credit	0.2
Split on term	0.25

Choice 1 vs Choice 2

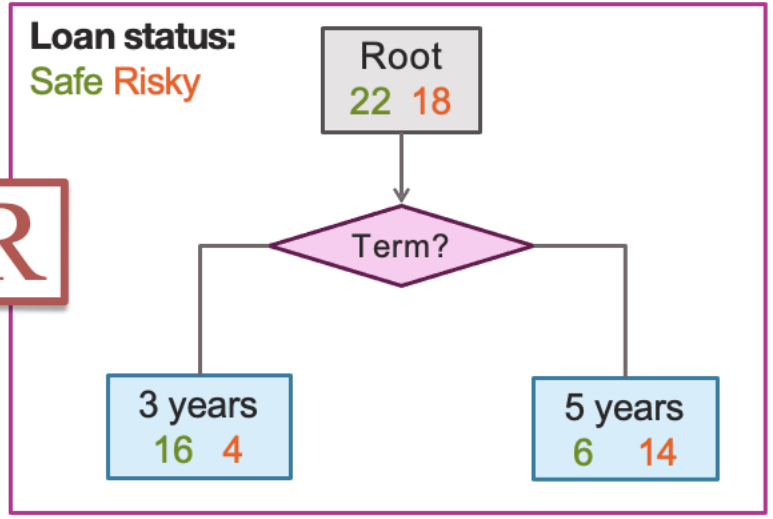
Tree	Classification error
(root)	0.45
Split on credit	0.2 √
Split on term	0.25

Choice 1: Split on Credit



OR

Choice 2: Split on Term



Feature split selection algorithm

- Given a subset of data M (a node in a tree)
- For each feature $h_i(x)$:
  1. Split data of M according to feature $h_i(x)$
  2. Compute classification error split
- Chose feature $h^*(x)$ with lowest classification error

Greedy Decision Tree Learning

- Step 1: Start with an empty tree
- Step 2: Select a feature to split data
- For each split of the tree:
  - Step 3: If nothing more to, make predictions
  - Step 4: Otherwise, go to Step 2 & continue (recurse) on this split

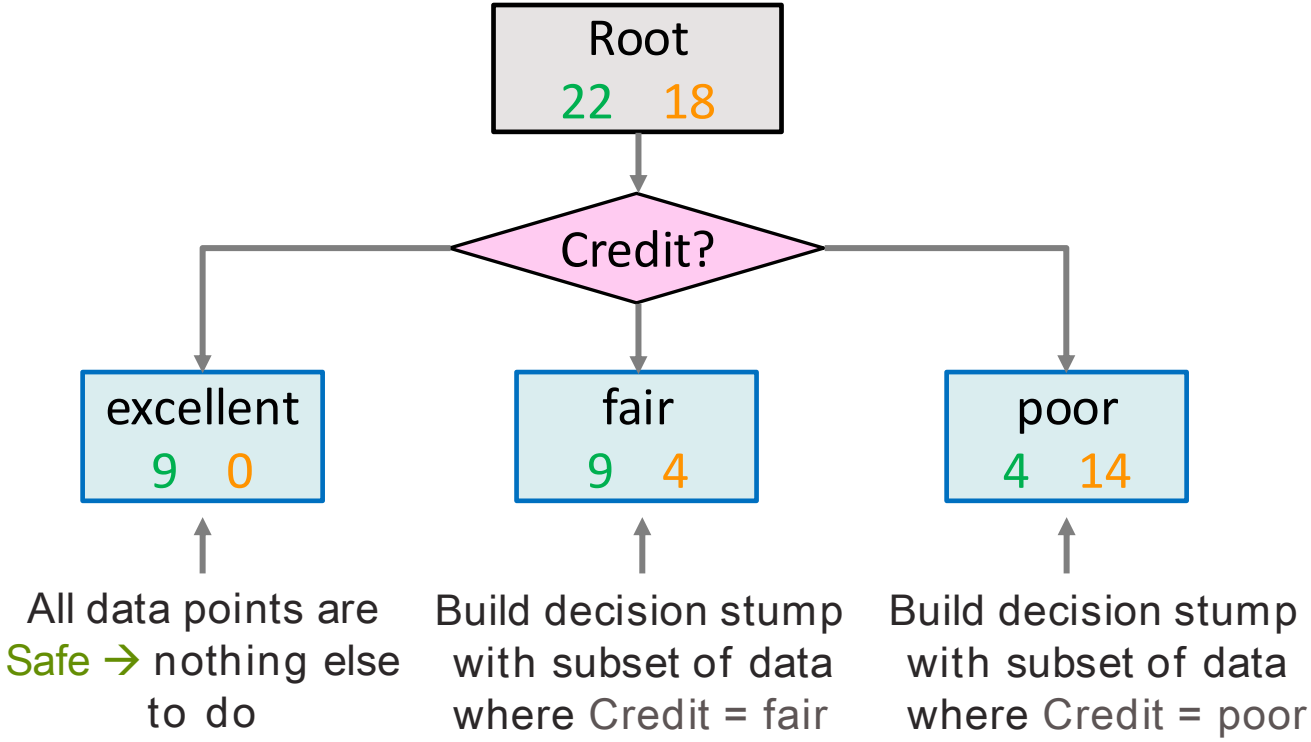
Problem 1: Feature split selection

Pick feature split leading to lowest classification error

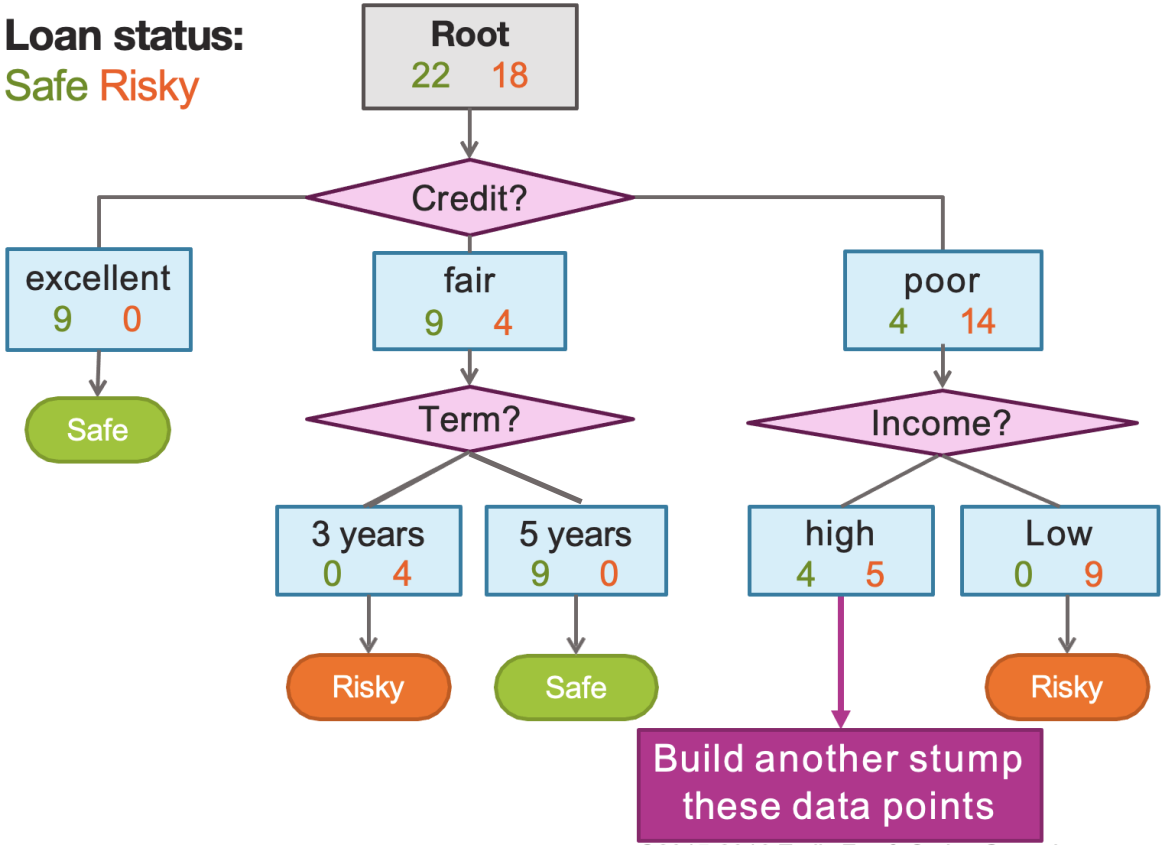
Problem 2: Stopping condition

Recursion

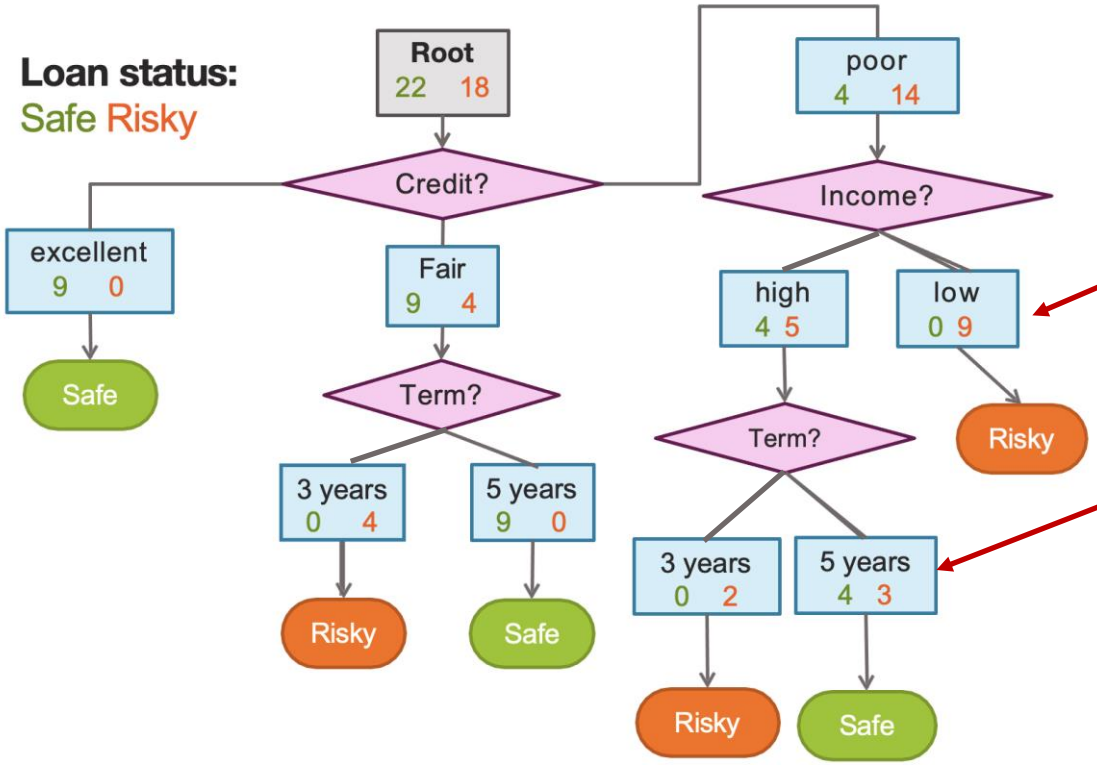
Stopping condition



Continue to split...



Stopping condition



All data in these nodes have same y value ➔ Nothing to do

Already split on all possible features ➔ Nothing to do

Greedy Decision Tree Learning

- Step 1: Start with an empty tree
- Step 2: Select a feature to split data
- For each split of the tree:
  - Step 3: If nothing more to, make predictions
  - Step 4: Otherwise, go to Step 2 & continue (recurse) on this split

Problem 1: Feature split selection

Pick feature split leading to lowest classification error

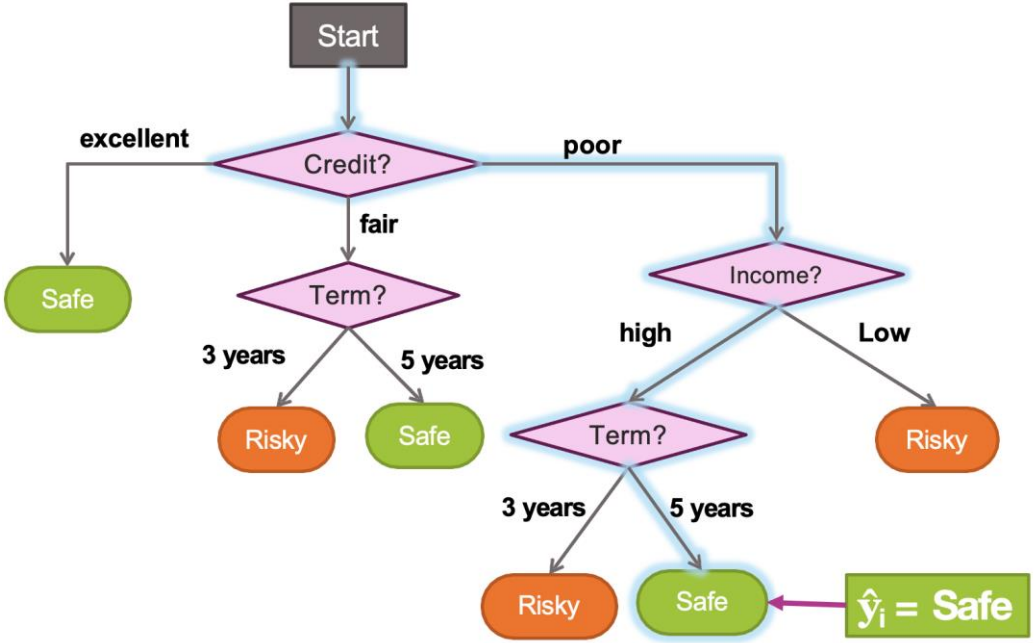
Problem 2: Stopping condition

Condition 1 & 2

Recursion

Decision tree prediction algorithm

x_i = (Credit = poor, Income = high, Term = 5 years)

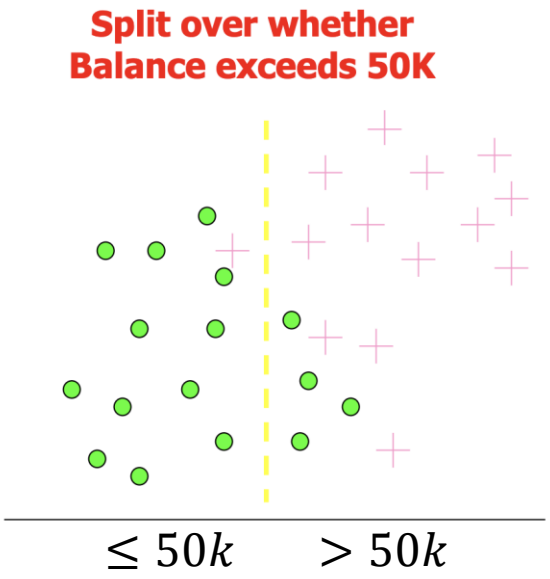


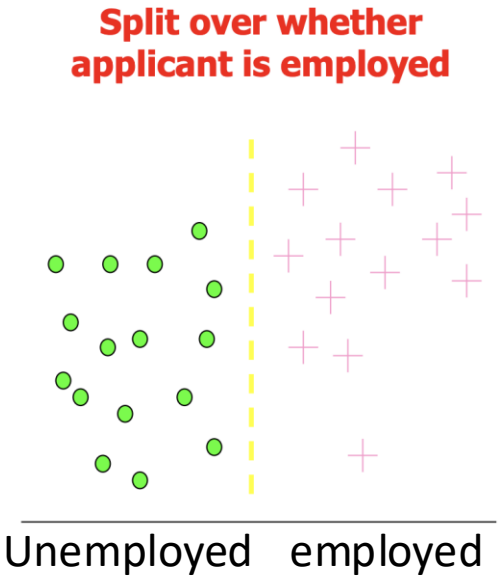
predict(tree_node, input)

- **If** current tree_node is a leaf:
  - **return** majority class of data points in leaf
- **else:**
  - next_note = child node of tree_node whose feature value agrees with input
  - **return predict**(next_note, input)

Another feature split selection criterion...

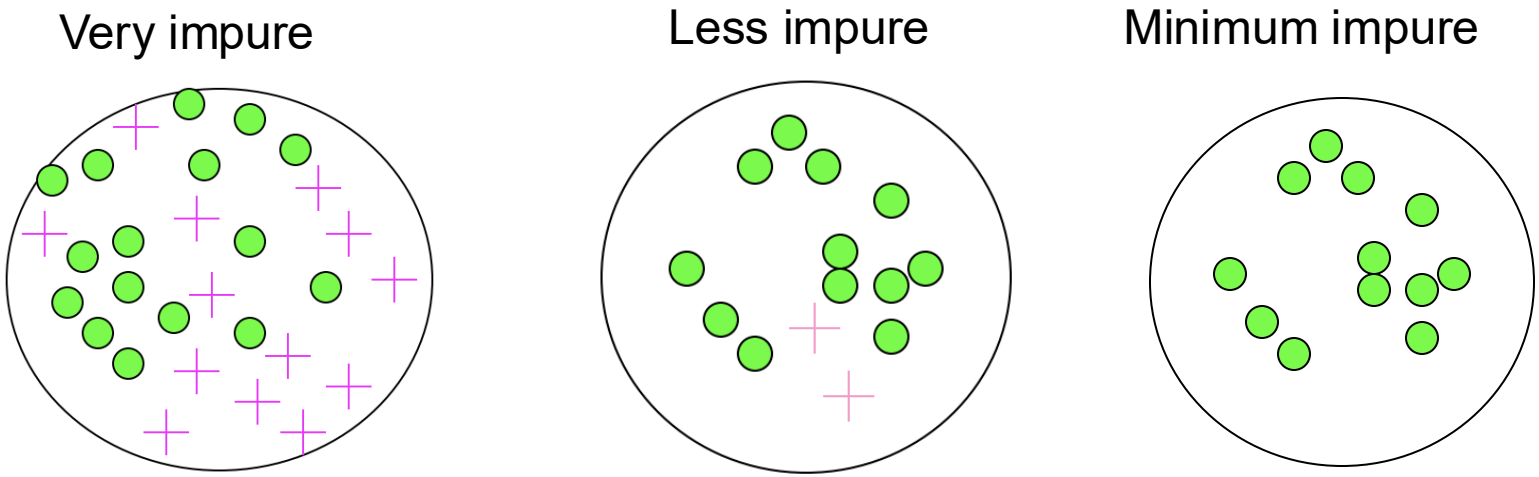
- Previously we had $$Error = \frac{\#incorrect\ predictions}{\#examples}$$





Information Gain

- Impurity/Entropy (informal)
  - Measures the level of **impurity** in a group of examples

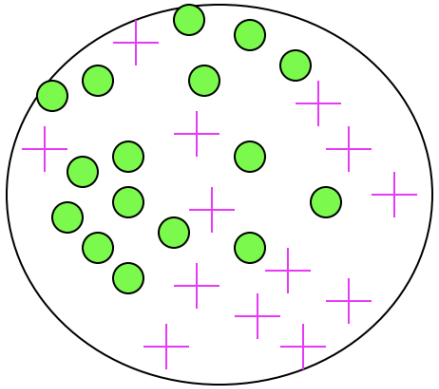


Entropy: a common way to measure impurity

- Entropy

$$H(x) = -\sum_{i=1}^{n} P(x = i)\log_2 P(x = i)$$

Probability of class i
Compute it as the proportion of class i in the set



$$H(x) = -\frac{16}{16+13}\log_2\frac{16}{16+13} - \frac{13}{16+13}\log_2\frac{13}{16+13}$$

$$= 0.9923$$

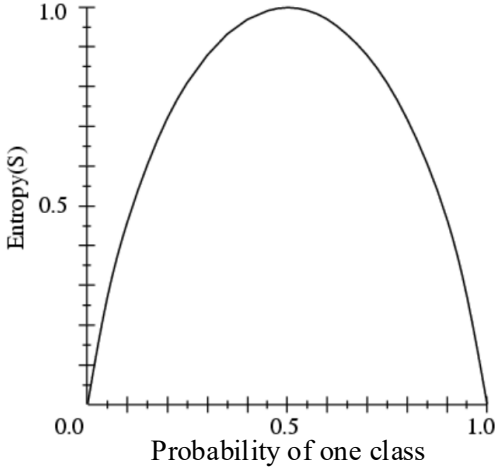
Entropy: a common way to measure impurity

- Entropy

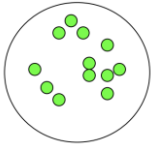
$$H(x) = -\sum_{i=1}^{n} P(x = i)\log_2 P(x = i)$$

Probability of class i
Compute it as the proportion of class i in the set

- 2-classes

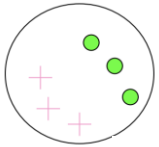


Minimum impurity



Entropy = $-1\log_2 1 - 0 = 0$

Maximum impurity



Entropy = $-0.5\log_2 0.5 - 0.5\log_2 0.5 = 1$

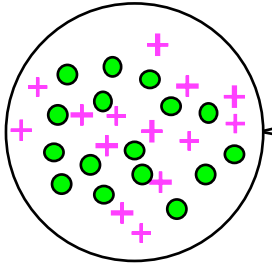
Information Gain

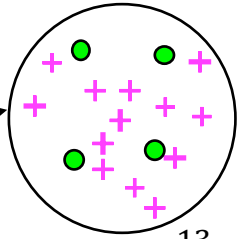
- We want to determine which attribute in a given set of training feature vectors is most useful for discriminating between the classes to be learned.
- **Information gain** tells us how important a given attribute of the feature vectors is.
- **Information gain =** entropy(parent) – [average entropy(children)]

Information Gain

Information gain = entropy(parent) – [average entropy(children)]

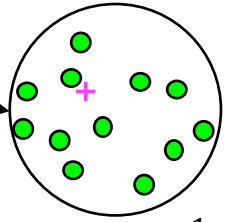
Entire Set
(30 instances)





17 instances

child entropy = $-(\frac{13}{17}\log_2\frac{13}{17})-(\frac{4}{17}\log_2\frac{4}{17}) = 0.787$



13 instances

Parent entropy $-(\frac{16}{30}\log_2\frac{16}{30})-(\frac{14}{30}\log_2\frac{14}{30}) = 0.996$

child entropy = $-(\frac{1}{17}\log_2\frac{1}{17})-(\frac{12}{17}\log_2\frac{12}{17}) = 0.391$

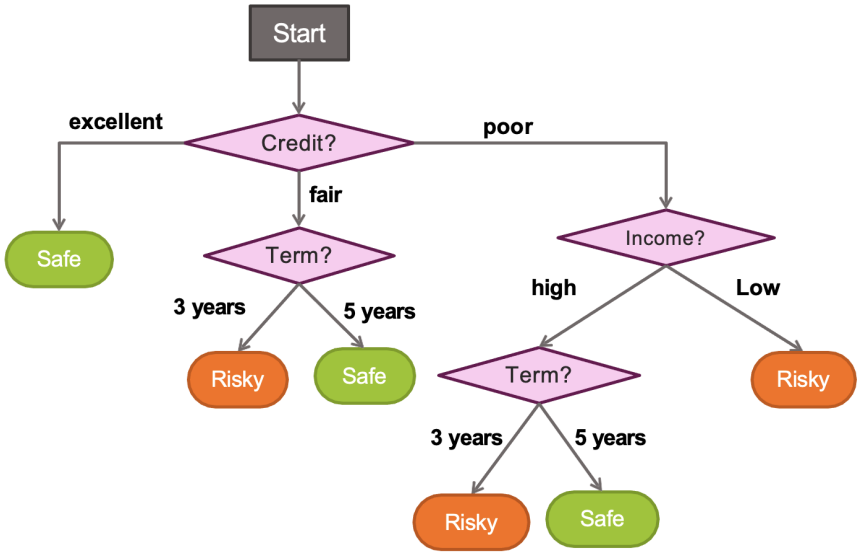
Weighted average entropy of children $= \frac{17}{30}0.787 + \frac{13}{30}0.391 = 0.615$

Information Gain $= 0.996 - 0.615 = 0.381$

Choose the feature with max Information Gain

Recap: Decision Tree

- Decision Tree Model
- Quality Metric
- Greedy decision tree learning
  - Feature split selection
  - Stopping condition

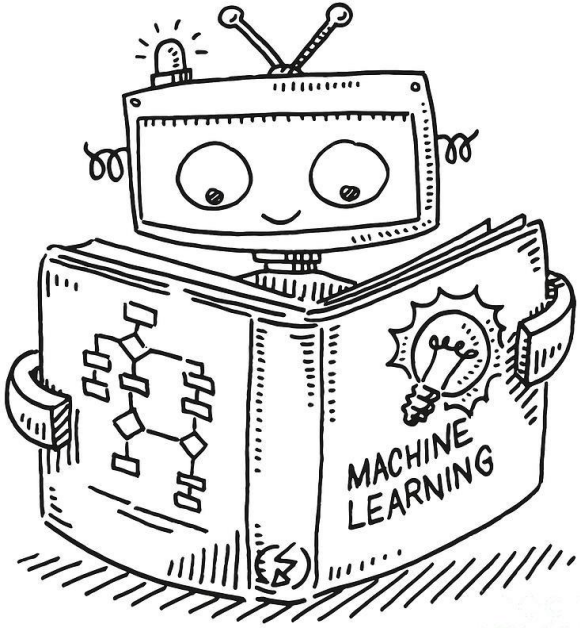


Assignment 2

- Choice 1: 我们已经学习了线性回归方法，在上一节课中，我们提到了（但没有深入解释）回归方法可能会存在过拟合（overfitting）的问题，请自行调研过拟合问题：什么是过拟合，以及解决过拟合有哪些具体的方法。请结合具体的例子解释方法过程，以报告的形式提交；
- Choice 2: 我们已经学习了逻辑回归、支持向量机、决策树这三种分类方法，本次课程中只提到了 +1/-1这种二分类，尚未介绍如何用这三种方法进行多分类。请自行调研多分类问题，如何将这些方法拓展到多分类问题上。请从三种方法中选择一种方法，例如，决策树，结合具体实例，阐述具体方法过程，以报告的形式提交。

此次作业内容将会作为下两周讨论课的主题

- 请各位同学积极报名展示自己的作业，分享自己的观点
- 将按照报名顺序选择前16名同学 （报名格式：学号-姓名-作业2，发送QQ消息）
- 每位学生仅有一次分享作业机会（还剩有3次机会/作业），请尽早报名

Today

- Classification: Support Vector Machine
- Classification: Decision Tree
- **Discussion**